

A surface-code simulator and decoder, checked against Stim, PyMatching and Google's hardware

Exact and correlated matching, Google's Willow and Sycamore data, and window decoding at Willow's cycle

Jasper Sands · 27 September 2026 · commit `3df0872`

`qcompiler.jaspersands.com` · `github.com/Jaspersands/quantum-simulator-qec` ·

SUMMARY

`stabilizer-qec` is a surface-code simulator and decoder written in Rust. It runs natively, from Python, and in a web browser. Its results come in five parts.

1. **Error models.** It builds error models identical to Stim's, fault by fault, on 15 circuits (the largest relative difference is $1.1e-15$).
2. **Matching.** Its exact matcher agrees with PyMatching on every shot but ties: 13 disagreements in 600,000 shots, 0 of them not ties. Its correlated matcher agrees with PyMatching 2.4's correlated mode just as closely, and fails 19% less often than plain matching.
3. **Google's hardware.** On Google's own recordings (550 experiments, 27.5 million shots) it rebuilds the detection events bit for bit. Its correlated matcher, using Google's SI1000 prior on Willow, reaches $\Lambda = 1.95$ [1.94, 1.95]. Google's own correlated matcher on the same prior reaches 1.91. Its belief-matching reproduces `ldpc`'s posteriors bit for bit, and makes Sycamore's $d = 5$ beat $d = 3$ as Google's does ($\Lambda = 1.016$ [1.01, 1.02]).
4. **Throughput.** Its bit-parallel sampler takes $0.84 \mu\text{s}$ a shot at $d = 7$, against Stim's $1.31 \mu\text{s}$.
5. **Real time.** Parallel windows decode Willow's recorded $d = 5$ syndromes with correlated matching and keep up with its $1.1 \mu\text{s}$ cycle on 4 cores. A simulated million-round stream at the same detection rate holds a $59 \mu\text{s}$ mean latency on 8 cores. Windowed decoding reaches the same Λ as global decoding (1.95 [1.94, 1.95]).

CONTENTS

- 1 Introduction
- 2 The engine
 - 2.1 Circuits and error models
 - 2.2 Sampling
 - 2.3 Matching
 - 2.4 Window decoding
 - 2.5 Where it runs
- 3 How it is checked
- 4 Results
 - 4.1 Thresholds
 - 4.2 Agreement with Stim and PyMatching
 - 4.3 Google's hardware

4.4	Belief-matching
4.5	Throughput
4.6	Real time
5	Limits and next steps
6	Reproducing everything

1 Introduction

A surface code protects a logical qubit by measuring checks on many physical qubits, round after round, and handing the outcomes to a decoder. The decoder infers which errors most likely occurred and whether they flipped the logical qubit. Two questions decide whether it is any good:

- **Is it correct?** Is its error model the circuit's, and is its matching exact?
- **Is it fast enough?** A quantum computer produces a round every microsecond or so, and a decoder that falls behind never catches up.

This report describes an engine built to answer both. It covers how the engine works, how it is checked, and what it measures. Every number below is read from a file committed with the source, and the build fails if one is missing. The same engine drives an interactive explainer that runs it in the reader's browser, at qcompiler.jaspersands.com.

The work is organised around three references the field already trusts:

- **Stim** (Gidney, 2021) for circuits, error models and sampling;
- **PyMatching 2** (Higgott and Gidney, 2023) for matching;
- **Google's published recordings** from Willow (2024) and Sycamore (2022) for real hardware.

Where this engine and a reference can be made to compute the same thing, they are compared exactly. Where they cannot, the comparison is statistical, with its uncertainty stated.

2 The engine

2.1 Circuits and error models

The engine reads and writes the part of Stim's circuit language that memory experiments use:

- resets, `H`, `CX` and `CZ`, and measurements;
- depolarizing and Pauli channels;
- detectors and observables with their coordinates, `SHIFT_COORDS`, and `REPEAT` blocks;
- for Google's circuits, sweep-controlled `CX` and the Pauli gates.

Two generators write rotated and XZZX memory experiments. One uses the engine's own circuit-level noise; the other uses SD6, the standard model in which published thresholds are quoted.

A **detector error model** lists every independent fault and the detectors and observables it flips. The engine builds one by walking the circuit backwards once. For each qubit it carries the set of detectors that an X or a Z error there would flip, as Stim's error analyzer does. Matching needs graph-like faults, so a fault that sets off more than two detectors is split into pieces. The engine splits them the way Stim does: by what the rest of the same noise channel can express, then by Stim's global pass. The first, simpler split looked right and was not (section 3).

2.2 Sampling

There are two samplers:

- **The frame sampler** propagates a Pauli frame through the circuit one shot at a time. It never looks at the error model, so the model and the sampler can disagree, and the checks can see it.
- **The batch sampler** keeps 64 shots in each machine word: two words per qubit, one for X and one for Z. It draws noise as whole words by geometric skipping, so a location at $p = 10^{-3}$ costs about one random number, not 64. It runs `REPEAT` blocks without flattening them, with measurement records in a ring buffer sized by the longest lookback. A million-round memory costs one round's instructions, and its detection events stream out as they are made.

2.3 Matching

Each graph edge is weighted $\ln((1-p)/p)$, discretised to even integers, and decoded by **sparse blossom**, the algorithm PyMatching 2 is built on (Higgott and Gidney, arXiv:2303.15933):

- Every defect grows a region on the detector graph at the same rate, and a region's radius is its dual variable.
- Collisions between regions are found by the growth itself.
- Edmonds' alternating trees and blossoms act on regions rather than on vertices.

The earlier dense matcher (Dijkstra from every defect, then blossom on the complete graph) stays in the code as the sparse one's first oracle.

Correlated matching follows PyMatching 2.4's `enable_correlations=True`, read from its source, so that PyMatching can serve as the oracle. A Y error sets off checks in both halves of the detector graph, and the error model records it as two pieces that fire together. Plain matching forgets that; correlated matching uses it in two passes:

1. **Match once.** Trace a shortest path for each matched pair into one edge set.
2. **Lower the weights.** For every edge in that set, lower the weight of each edge it shares a fault with to $\ln((1-p_a)/p_a)$. Here $p_a = \min(1/2, \text{joint}(c, a)/\text{marginal}(c))$ is the probability that edge a fired, given that edge c did.
3. **Match again** on the lowered weights, and read off the prediction.

2.4 Window decoding

A global decoder waits for the experiment to end. A **window decoder** matches a few rounds at a time:

- It decodes a window of *commit* + *buffer* rounds and keeps the edges of its correction that touch the commit region.
- It toggles their far ends, so a correction reaching past the region leaves a defect for the next window.
- An edge leaving the window becomes a boundary half-edge of its own, kept apart from the real boundary. A defect near the window's end can then wait for a partner not yet seen.

There are two schedules:

- **Sliding** windows run one after another, so each stream uses one core.
- **Parallel** windows (Skoric et al., arXiv:2209.08552; Tan et al., arXiv:2209.09219) come in two layers. Layer A's windows are spaced apart, with virtual boundaries on both sides. Layer B's fill the gaps once A has committed. Each layer's windows are independent, so one stream can use many cores.

Correlated matching works inside a window: the second pass is traced on the lowered weights, and the correlation rules are restricted to the window's edges.

Streaming. A million-round experiment has no error model that fits in memory, and none is needed: a memory circuit repeats. The stream decoder builds the model of a short *template* once, and gives each window of the stream the graph of the template window in the same position. The first and last windows come from the template's ends, the bulk from its middle. Rounds are pushed in as the batch sampler makes them, and a window is decoded as soon as its last round has arrived and the windows it depends on are done. Once no window still needs a round, the round is dropped, and any defect left in it is counted as unexplained.

2.5 Where it runs

The same Rust runs natively, from Python (a PyO3 module, `pip`-installable as one abi3 wheel for Python 3.9 on), and as WebAssembly in the browser. The page has no build step. It runs the engine in a pool of web workers, one per core but one, at most eight. The engine is compiled once and shared by every worker.

3 How it is checked

The checks come in three layers, strictest first:

- **Exact oracles.** Where an answer is unique, it is checked exactly:
 - The sparse matcher's optimal weight must *equal* the dense matcher's and a brute-force search's, on random graphs and on sampled surface-code shots.
 - Every single fault of every circuit, decoded alone, must predict its own logical flip.
 - One window as long as the stream must reproduce the global decoder's traced correction.
 - The batch sampler must reproduce the frame sampler's shots when every channel is made deterministic.
- **References.** Stim's error models must equal ours, and PyMatching's matchings must tie ours. Google's detection events must equal those we rebuild from their raw measurements.
- **Statistics.** Where only a rate can be compared, the comparison states its interval: per-detector χ^2 between samplers, Wilson intervals on logical error rates, and parametric bootstrap intervals on Λ .

The layers have caught real defects. They are listed in full in the repository's README, and four of them show what each layer is for:

- **A decomposition that passed every unit test.** It split wide faults into their X and Z halves. It agreed with Stim fault for fault, because that comparison is about faults, not pieces. But the exhaustive single-fault check found faults at $d = 3$ that it decoded into logical errors: a Y error on a boundary qubit, kept whole, became an edge bridging the two matching graphs. The fix reproduces Stim's decomposition exactly, and the matching graphs in section 4.2 are the result.
- **Refitting that biased the intervals.** A bootstrap that let each redrawn data set re-choose its own points and weights biased the redrawn fidelities about 0.2% low. Averaged over eighteen patches, that made the intervals lopsided. It was caught before anything was quoted: the intervals had come out asymmetric where they should not have.
- **Parallel windows with no buffer.** They were found to leave corrections nowhere to land. The every-defect-explained check found them, and they are now refused.
- **A stream count that could not fail.** Code review found that the stream decoder dropped rounds without counting the defects left in them, so its "unexplained" count was zero by construction. It now counts them. A

test that skips a window shows the leftover defect counted (and fails on the old code), and the million-round run below was repeated with the working count.

4 Results

4.1 Thresholds

SD6 thresholds from repeated sweeps over $d = 3$ to 9, each fitted by finite-size scaling. The fit includes the leading correction to scaling, over the widest window the scaling form describes, chosen by reduced χ^2 (4 independent sweeps per code; `data/sweeps/`):

code	p_th (mean $\pm$ spread)	v	uncorrected crossing	sweeps
rotated	0.54% $\pm$ 0.01	1.11	0.48%	4
XZZX	0.51% $\pm$ 0.01	1.22	0.47%	4

The “uncorrected crossing” is where the curves cross without the correction term, which small patches pull low; the corrected fit is the threshold.

4.2 Agreement with Stim and PyMatching

Error models. The comparison covers Stim’s own generated rotated memory, and ours for both codes under both noise models, at $d = 3, 5$ and 7 with $p = 0.3\%$. For all of them, both models list the same faults with the same probabilities, and the decomposed matching graphs are identical edge for edge:

circuit	detectors, $d = 3 / 5 / 7$	mechanisms	graph edges	largest $\Delta p/p$	vs Stim
Stim’s rotated_memory_z	24 / 120 / 336	219 / 1,677 / 5,471	78 / 502 / 1,558	9.9e-16	identical
rotated, engine model	36 / 156 / 408	223 / 1,261 / 3,739	103 / 581 / 1,723	2.2e-16	identical
rotated, SD6	24 / 120 / 336	219 / 1,677 / 5,471	78 / 502 / 1,558	1.1e-15	identical
XZZX, engine model	36 / 156 / 408	223 / 1,261 / 3,739	103 / 581 / 1,723	2.9e-16	identical
XZZX, SD6	24 / 120 / 336	219 / 1,677 / 5,471	78 / 502 / 1,558	1.1e-15	identical

Plain matching. Stim sampled 100,000 shots at each point, and both decoders decoded the same detection events. Two exact matchers can disagree only where two corrections tie in weight. There were 13 disagreements in 600,000 shots, and every one is a tie: the two matchings’ weights are equal to within the discretisation noise.

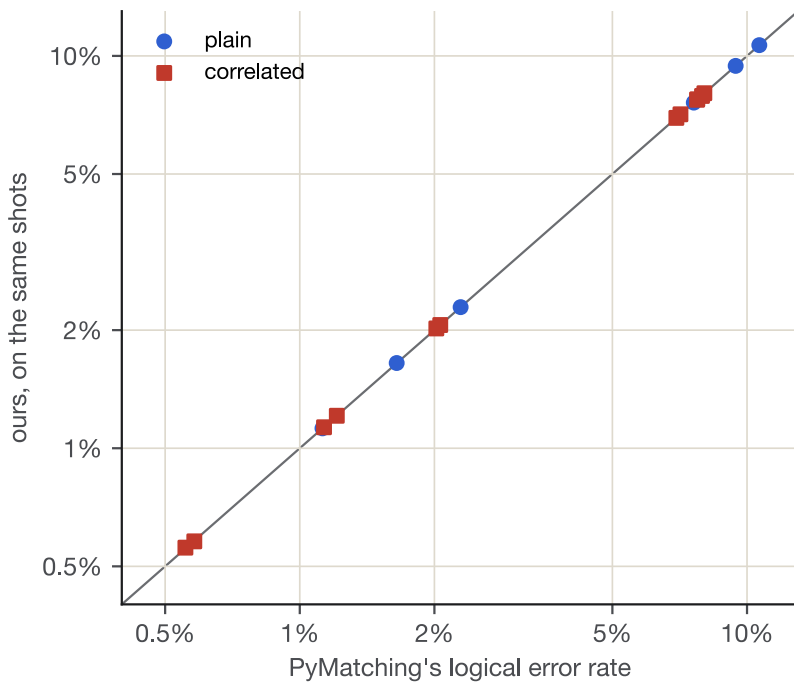
d	p	PyMatching	ours	ours, own model	disagreements	non-ties	PyMatching μ s	ours μ s
3	0.3%	2.289%	2.289%	2.289%	0	0	0.2	0.6
3	0.6%	7.603%	7.603%	7.603%	0	0	0.4	1.2
5	0.3%	1.647%	1.649%	1.649%	2	0	1.6	4.3
5	0.6%	9.432%	9.432%	9.432%	4	0	4.0	9.8
7	0.3%	1.124%	1.124%	1.124%	0	0	5.7	15.1
7	0.6%	10.653%	10.652%	10.652%	7	0	14.2	36.3

Correlated matching, on 1,200,000 shots: 696 disagreements, and none of them a bug. For each, PyMatching's own first-pass edges went into our second pass:

- in 682 cases this reproduces PyMatching's answer, because the two first passes traced different, equally short paths;
- the other 14 tie PyMatching's second-pass weight;
- 0 are neither.

Correlated matching fails 53,030 times against plain matching's 65,316, 19% fewer.

code	d	p	PyMatching	ours	ours, plain	disagreements	path ties	weight ties	non-ties
rotated	3	0.3%	2.061%	2.061%	2.332%	0	0	0	0
rotated	3	0.6%	6.951%	6.951%	7.708%	0	0	0	0
rotated	5	0.3%	1.133%	1.131%	1.645%	8	7	1	0
rotated	5	0.6%	7.735%	7.738%	9.335%	31	27	4	0
rotated	7	0.3%	0.555%	0.558%	1.037%	29	29	0	0
rotated	7	0.6%	7.932%	7.906%	10.590%	274	270	4	0
XZZX	3	0.3%	2.020%	2.020%	2.293%	0	0	0	0
XZZX	3	0.6%	7.098%	7.098%	7.665%	0	0	0	0
XZZX	5	0.3%	1.210%	1.210%	1.639%	8	8	0	0
XZZX	5	0.6%	7.739%	7.745%	9.323%	22	22	0	0
XZZX	7	0.3%	0.581%	0.579%	1.103%	32	32	0	0
XZZX	7	0.6%	8.027%	8.033%	10.646%	292	287	5	0



Our logical error rate against PyMatching's on the same shots: plain matching at six points and correlated matching at twelve, SD6, 100,000 shots each. Every point lies on the diagonal.

Speed. Single-threaded, our plain matcher takes 2.5 to 2.9 times PyMatching's time per shot, and our correlated matcher 2.0 to 2.2 times PyMatching's correlated mode. PyMatching's C++ has had years of tuning that this matcher has not. The gap is a roughly constant factor that does not grow with d , as the figure below shows.

4.3 Google's hardware

Google has published everything its chips recorded in two surface-code memory experiments:

- **Willow** (2024): $d = 3, 5$ and 7 , 1 to 250 rounds;
- **Sycamore** (2022): $d = 3$ and 5 , 1 to 25 rounds.

The data comes as raw measurements, sweep bits, circuits, error models and each of Google's decoders' predictions. Both datasets are CC BY 4.0, from Zenodo records 13273331 and 6804040.

Reading the chips' output. A detector compares a parity of measurements with what a noiseless run gives. The engine runs the ideal circuit once on its stabilizer tableau to get that reference, and once more per sweep bit to get each bit's effect. This is independent of the backward walk that builds error models. Over all 550 experiments and 27.5 million shots:

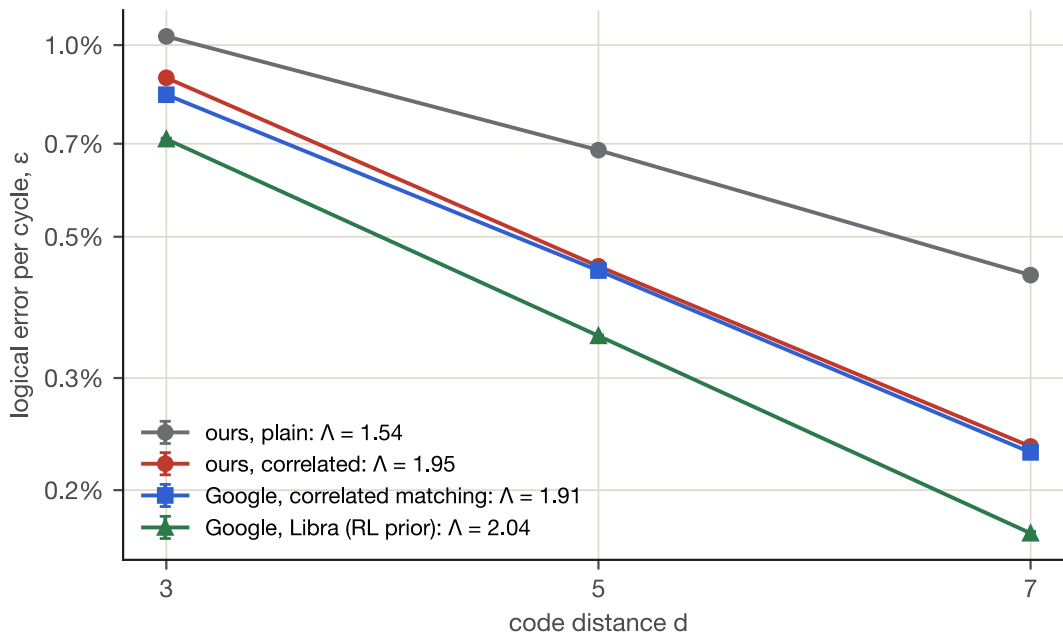
- the detection events and observable flips rebuilt from raw measurements equal Google's **bit for bit** (550 of 550 experiments);
- our model of every noisy circuit is identical to Stim's (550 of 550).

One fit, Google's. Λ is fitted the way Google fits it:

1. **Per patch and basis**, the logical fidelity $1 - 2P_L$ is fitted as $A(1 - 2\varepsilon)^f$ by weighted least squares on its logarithm, from round 10 for Willow and round 3 for Sycamore.
2. **Per distance**, ε is the mean over patches and bases.
3. Λ is $\exp(-2 \text{slope})$ of a line through $\ln \varepsilon$ against d .
4. **Intervals** come from a parametric bootstrap with 400 draws.

As validation, Google's own tensor-network predictions for Sycamore, fitted this way, give $\epsilon_3 = 3.028\%$ and $\epsilon_5 = 2.914\%$. The published values are 3.028% and 2.914%: the same, to the digits published.

decoder	prior	ϵ , d = 3	ϵ , d = 5	ϵ , d = 7	Λ [95%]
ours, plain	SI1000	1.032%	0.684%	0.435%	1.54 [1.53, 1.55]
ours, correlated	SI1000	0.888%	0.449%	0.234%	1.95 [1.94, 1.95]
ours, correlated	RL-optimised	0.808%	0.425%	0.222%	1.91 [1.90, 1.92]
ours, correlated	ours, from the circuit	0.884%	0.468%	0.241%	1.91 [1.91, 1.92]
Google, correlated matching	SI1000	0.836%	0.443%	0.229%	1.91 [1.90, 1.92]
Google, correlated matching	RL-optimised	0.759%	0.404%	0.210%	1.90 [1.90, 1.91]
Google, Harmony	RL-optimised	0.729%	0.387%	0.206%	1.88 [1.88, 1.89]
Google, Libra	RL-optimised	0.712%	0.349%	0.171%	2.04 [2.03, 2.05]



Willow's logical error per cycle against code distance, from every experiment's recorded shots, fitted as Google fits them. Bars are 95% bootstrap intervals.

- **Correlated matching is most of the story.** With the same prior, it takes Λ from 1.54 to 1.95, and ϵ at $d = 7$ from 0.435% to 0.234%.
- **Ours sits beside Google's own correlated matcher** on the same prior, 0.234% against 0.229% at $d = 7$. It is further behind at $d = 3$ (0.888% against 0.836%), which is why its Λ comes out higher: Λ rewards improving with size, not being good.
- **Our own model of the noisy circuit**, built from Google's circuit with no fitting to the data, does about as well as Google's fitted SI1000 prior (0.241% at $d = 7$).

- **Google’s best decoders are better still.** Libra reaches $\Lambda = 2.04$ and $\varepsilon_r = 0.171\%$. The published neural-network decoder’s 2.14 cannot be refitted here: its predictions are not in the dataset.

decoder	prior	$\varepsilon, d = 3$	$\varepsilon, d = 5$	Λ [95%]
ours, plain	circuit	4.012%	4.354%	0.92 [0.91, 0.93]
ours, correlated	circuit	3.507%	3.558%	0.99 [0.98, 0.99]
ours, correlated	p _{ij} , cross-fitted	3.424%	3.466%	0.99 [0.98, 1.00]
Google, PyMatching	circuit	4.012%	4.362%	0.92 [0.91, 0.93]
Google, correlated matching	circuit	3.497%	3.597%	0.97 [0.96, 0.98]
Google, belief matching	p _{ij}	3.118%	3.056%	1.02 [1.01, 1.03]
Google, tensor network	p _{ij}	3.028%	2.914%	1.04 [1.03, 1.05]

On Sycamore, our plain matcher reproduces the PyMatching predictions Google recorded (4.012% against 4.012% at $d = 3$). The two disagree on 8,448 shots over the whole dataset. On every one of them, our matching’s weight equals the optimum PyMatching 2.4 finds (0 are not optimal): these are ties, broken differently by the older PyMatching that Google used. Sycamore’s $\Lambda \approx 1$ was that paper’s point, the first time a larger surface code beat a smaller one at all, and only with the best decoders.

4.4 Belief-matching

Matching weighs every graph edge by its prior alone. **Belief-matching** (Higgott, Bohdanowicz, Kubica, Flammia and Campbell, PRX 13, 031007, 2023) works in three steps:

1. **Belief propagation** runs over the whole error model: every fault a variable, every detector a check, nothing split into pieces.
2. **The graph is re-weighted.** Each edge gets $-\ln p$, where p is the sum of the posteriors of the faults whose decomposition contains it.
3. **The sparse matcher decodes** on those weights. Where BP converges by itself, its own correction is the answer.

The implementation is written to reproduce the authors’ package, `beliefmatching`, and the BP library it runs on, `ldpc`, step for step. Both are checked exactly:

- **BP.** Over 4,040 syndromes in 56 configurations, the largest difference between our posterior log-likelihood ratios and `ldpc`’s is 0.0. The configurations cover random matrices and a surface code’s hypergraph, product-sum and min-sum, and 1 to 20 iterations. 0 hard decisions, convergence flags or iteration counts differ.
- **Belief-matching.** Over 20,000 shots of SD6 memories at $d = 3, 5$ and 7 , it disagrees with `beliefmatching` on 0 shots, and BP’s convergence differs on 0. Single-threaded at $d = 7$ it takes 5.6 ms against 11.6 ms a shot; the reference rebuilds a matching graph for every shot.

Sycamore, every shot, with the data-fitted priors cross-fitted as Google used them, against Google’s own recorded belief-matching:

decoder	prior	ϵ , d = 3	ϵ , d = 5	Λ [95%]
ours, belief-matching	pjj, cross-fitted	3.154%	3.104%	1.02 [1.01, 1.02]
ours, belief-matching	circuit	3.249%	3.190%	1.02 [1.01, 1.03]
Google, belief matching	pjj	3.118%	3.056%	1.02 [1.01, 1.03]
Google, tensor network	pjj	3.028%	2.914%	1.04 [1.03, 1.05]
Google, correlated matching	circuit	3.497%	3.597%	0.97 [0.96, 0.98]

- **d = 5 beats d = 3.** Ours gives $\Lambda = 1.016$ [1.01, 1.02], Google's 1.020 [1.01, 1.03].
- **Close, but not the same.** Ours agrees with Google's predictions on 97.2% of 6.5 million shots and fails 0.7% more often. Google's BP settings are not in the dataset, and more iterations do not close the gap.

Willow, on the first 10,000 shots of each of 420 experiments. BP on Willow's longest experiments costs about 25 ms a shot even on ten cores, so there are fewer shots. Every decoder in this table is scored on the same shots:

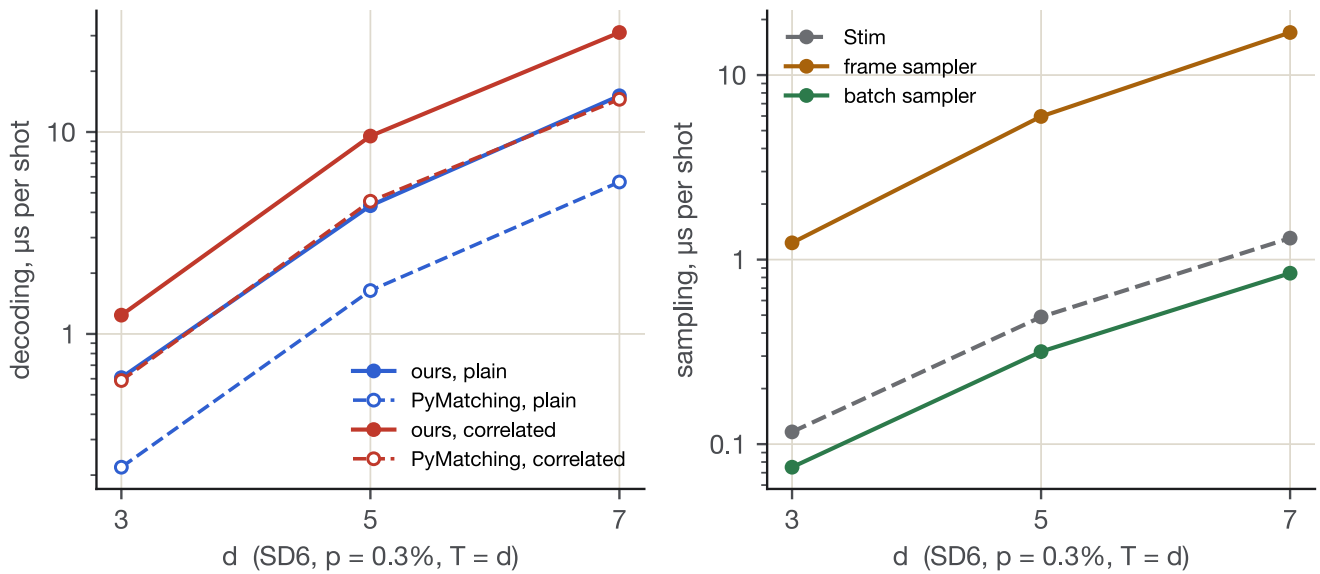
decoder, same shots	ϵ , d = 3	ϵ , d = 5	ϵ , d = 7	Λ [95%]
ours, belief-matching	0.824%	0.452%	0.251%	1.81 [1.80, 1.83]
ours, correlated matching	0.888%	0.448%	0.231%	1.96 [1.95, 1.98]
ours, plain matching	1.028%	0.675%	0.429%	1.55 [1.54, 1.56]
Google, correlated matching	0.835%	0.442%	0.229%	1.91 [1.90, 1.92]
Google, Libra (RL prior)	0.713%	0.350%	0.170%	2.05 [2.03, 2.06]

- **Belief-matching wins small and loses large.** At d = 3 it beats every matcher in the table (0.824%, against 0.888% for our correlated matcher and 0.835% for Google's). At d = 7 it is worse (0.251% against 0.231%), so its Λ , 1.81 [1.80, 1.83], falls below correlated matching's 1.96 [1.95, 1.98].
- **Longer BP does not change it.** On the same shots at d = 7, 50 rounds, belief-matching fails 960 times with 20 iterations and 959 with 100, against correlated matching's 902. BP settles alone on 16% of Willow's shots.
- **On Sycamore it is among the best.** Those experiments run at most 25 rounds with priors fitted to the data, and belief-matching is one of the two best decoders there, Google's and ours alike.

4.5 Throughput

Sampling speed per shot, single-threaded, rotated SD6 with $T = d$. Stim is timed through its Python API, compiled beforehand and writing packed bits; ours includes transposing batches into Stim's b8 layout:

d	p	Stim	frame sampler	batch sampler	Stim ÷ batch
3	0.3%	0.12 μ s	1.23 μ s	0.07 μ s	1.6×
3	0.6%	0.19 μ s	1.27 μ s	0.09 μ s	2.1×
5	0.3%	0.49 μ s	6.0 μ s	0.32 μ s	1.5×
5	0.6%	0.89 μ s	6.2 μ s	0.40 μ s	2.2×
7	0.3%	1.31 μ s	17.0 μ s	0.84 μ s	1.5×
7	0.6%	2.3 μ s	17.5 μ s	1.06 μ s	2.1×



Decoding (left) and sampling (right) per shot against distance, single-threaded, SD6 at $p = 0.3\%$.

The batch sampler is 14 to 20 times as fast as the frame sampler, and faster than Stim's as called from Python. Decoding, not sampling, is now almost all of a shot's cost.

On the page, work is split across a pool of workers. On the development machine the site's section 7 threshold sweep ran 2.5 to 4.4 times faster with the pool than with one worker; the spread comes from whether workers land on performance or efficiency cores.

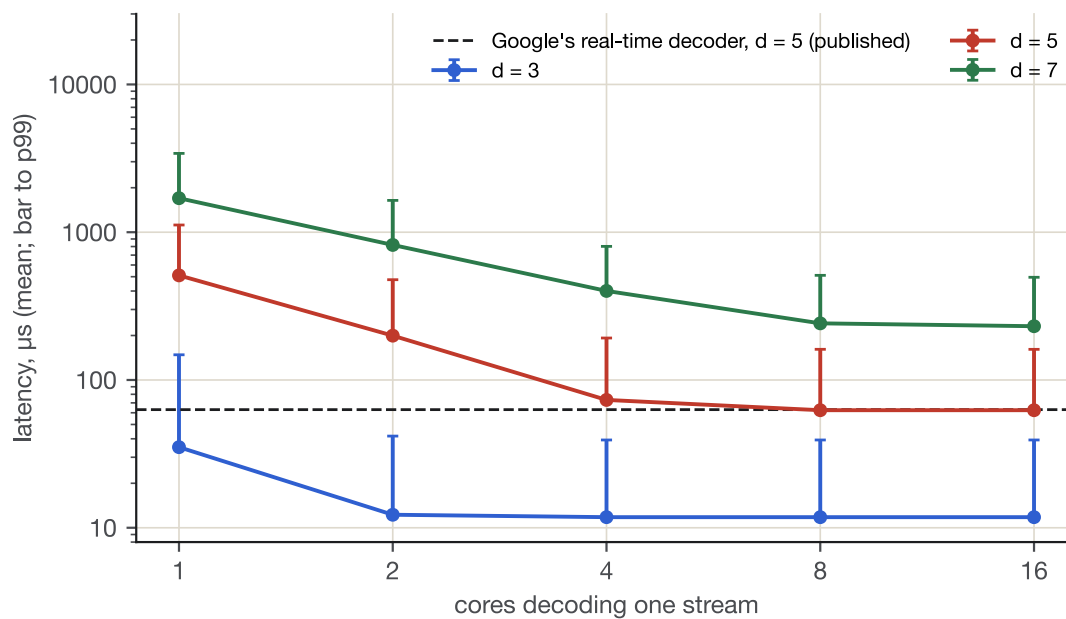
4.6 Real time

Willow runs a round of error correction every $1.1 \mu\text{s}$. Google has decoded a $d = 5$ memory in real time over a million rounds with a $63 \mu\text{s}$ mean latency (arXiv:2408.13687), on its own hardware and with its own definition of latency. Here, every window's decode time is measured natively on Apple M2 Pro, one core at a time. Those times are then scheduled with rounds arriving every $1.1 \mu\text{s}$:

- A window starts once its last round has arrived and, for layer B, both its layer-A neighbours are done.
- **Latency** is the time from the arrival of a window's last round to its commit.
- A stream **keeps up** if its latency does not grow along it.

The windows commit d rounds with a buffer of d . The streams are Willow's recorded syndromes, 2,000 shots at each of $d = 3, 5$ and 7 , at 250 rounds, decoded with Google's SI1000 prior:

stream	windows	matcher	window decode	keeps up on	mean latency	p99
d = 3	sliding	plain	2.1 μ s	1 core	4.4 μ s	27 μ s
d = 3	parallel	plain	2.5 μ s	1 core	7.9 μ s	28 μ s
d = 3	sliding	correlated	4.0 μ s	no (one core by design)	–	–
d = 3	parallel	correlated	5.2 μ s	2 cores	12.2 μ s	42 μ s
d = 5	sliding	plain	13.9 μ s	no (one core by design)	–	–
d = 5	parallel	plain	16.9 μ s	4 cores	33 μ s	86 μ s
d = 5	sliding	correlated	27 μ s	no (one core by design)	–	–
d = 5	parallel	correlated	34 μ s	4 cores	73 μ s	192 μ s
d = 7	sliding	plain	50 μ s	no (one core by design)	–	–
d = 7	parallel	plain	62 μ s	8 cores	111 μ s	235 μ s
d = 7	sliding	correlated	106 μ s	no (one core by design)	–	–
d = 7	parallel	correlated	133 μ s	8 cores	242 μ s	511 μ s



Mean window latency (points; bars reach the 99th percentile) against the cores given to one stream, parallel windows with correlated matching on Willow's recorded 250-round syndromes.

- **Sliding windows** keep up only at $d = 3$ with plain matching. Everywhere else one core is too slow, which is why parallel windows exist.
- **At $d = 5$, parallel windows with correlated matching keep up on 4 cores** with a 73 μ s mean latency. At $d = 7$ they need 8.

A million rounds. The stream is a rotated $d = 5$ memory under SD6 at $p = 0.313\%$, the noise at which its detectors fire as often as Willow's do (7.5% of detectors a round). There are 64 streams of 1,000,000 rounds each, sampled round by round and decoded as they arrive:

windows	matcher	streams	windows each	unexplained	window decode	cores	mean latency	p99
sliding	plain	64	200,000	0	12.1 μ s	—	—	—
parallel	plain	64	133,334	0	15.5 μ s	2	52 μ s	193 μ s
sliding	correlated	64	200,000	0	25 μ s	—	—	—
parallel	correlated	64	133,334	0	32 μ s	4	117 μ s	421 μ s

- **Every defect of every stream is explained** (0 left, counted as rounds are dropped).
- **Correlated matching on 8 cores holds a 59 μ s mean latency** (137 μ s at the 99th percentile) over a million rounds.
- **Throughput.** On all 10 cores, independent streams decode at 3.2 million rounds a second with plain matching and 1.6 million with correlated. Willow's cycle demands 0.91 million a second per stream.

Accuracy. Windowing costs nothing in Λ . The run covers 392 Willow experiments (all but the single-round ones, which have nothing to window), 50,000 shots each. Every one was decoded globally and by sliding and parallel windows, fitted as above:

decoder	matcher	buffer	ϵ , d = 3	ϵ , d = 5	ϵ , d = 7	Λ [95%]
global	plain	—	1.032%	0.684%	0.435%	1.54 [1.54, 1.55]
sliding	plain	d	1.034%	0.685%	0.436%	1.54 [1.53, 1.54]
parallel	plain	d	1.033%	0.685%	0.436%	1.54 [1.53, 1.54]
parallel	plain	d/2 (d = 5 only)	—	0.694%	—	—
parallel	plain	2d (d = 5 only)	—	0.684%	—	—
global	correlated	—	0.888%	0.449%	0.234%	1.95 [1.94, 1.95]
sliding	correlated	d	0.892%	0.454%	0.236%	1.94 [1.94, 1.95]
parallel	correlated	d	0.892%	0.452%	0.235%	1.95 [1.94, 1.95]
parallel	correlated	d/2 (d = 5 only)	—	0.460%	—	—
parallel	correlated	2d (d = 5 only)	—	0.451%	—	—

- **Correlated matching gives the same Λ either way:** 1.95 [1.94, 1.95] for parallel windows, 1.95 [1.94, 1.95] globally.
- **A buffer of d/2 is too short.** It raises ϵ_s by 1.8%, while a buffer of 2d gains nothing over d.
- **Simulated SD6 agrees.** At d = 3, 5 and 7, $p = 0.3\%$ and 0.5% , 50 rounds and 20,000 shots, every window decoder's failures are within 1.6% of the global decoder's on the same shots.
- **No defect was left unexplained** (0 over every Willow run).

5 Limits and next steps

- **The matcher.** It is exact but not the fastest: it runs at 2.5 to 2.9 times PyMatching's single-threaded time. Correlated matching closes most of the gap to Google's correlated matcher, but Google's Harmony and Libra decoders do better still.
- **Latency is scheduled, not served.** The latency figures schedule decode times measured natively on one machine. They leave out the transport of syndromes from a quantum computer to the decoder, and they are not

a claim about any particular control system.

- **The next decoders.** The next pieces of work, in order:
 - the IBM “gross” bivariate-bicycle code with BP+OSD, reusing this belief propagation;
 - lattice surgery between patches;
 - a resource estimator driven by the Λ measured here.

6 Reproducing everything

Every number in this report is filled in by `tools/report.py` from these committed files:

- `data/xcheck/reference.json` : the cross-check;
- `data/sweeps/` : thresholds;
- `data/google-results/` : Google’s data, decoded;
- `data/realtime/` : latency, the million-round stream, and windowed accuracy.

The report fails to build if a value it quotes is missing. To regenerate the data:

```
python3 -m venv .venv && .venv/bin/pip install stim pymatching numpy maturin
VIRTUAL_ENV=$PWD/.venv CARGO_TARGET_DIR=target-py .venv/bin/maturin develop --profile python
.venv/bin/python tools/xcheck.py                # Stim and PyMatching
.venv/bin/python tools/google.py run             # Google's data (fetch it first)
.venv/bin/python tools/realtime.py latency       # window latency on Willow
.venv/bin/python tools/realtime.py million       # a million rounds
.venv/bin/python tools/realtime.py accuracy     # windowed against global
node tools/lambda.mjs                           # every fit
python3 tools/report.py                         # this report
```

The package installs with `pip install stabilizer-qec` once published, or from a wheel built by `maturin build`. Software: Stim 1.16.0, PyMatching 2.4.0. Google’s datasets are CC BY 4.0 (Google Quantum AI; Zenodo 13273331 and 6804040); the engine is MIT-licensed.